# Функции в Python

# Основы работы с функциями

Функция — это самостоятельный, поименованный блок кода, который выполняет определенную задачу. Подобно тому, как на кухне мы используем различные инструменты для разных операций (нож для нарезки, венчик для взбивания), функции позволяют нам создавать собственные «инструменты» для решения повторяющихся задач в программе.

# Основы работы с функциями

Представьте, что в программе требуется несколько раз выполнить одно и то же действие — например, проверить, является ли строка палиндромом, или вычислить факториал числа. Без функций нам пришлось бы копировать один и тот же код в разные места программы. Это приводит к ряду проблем :

- *Дублирование кода* — программа становится громоздкой
- *Сложность поддержки* — если нужно изменить логику, придется править код во всех местах
- *Снижение читаемости* — длинный код труднее понимать

Функции решают эти проблемы, предоставляя следующие преимущества :

- *Модульность* — разбиение программы на логические блоки
- *Повторное использование* — написанный однажды код можно вызывать многократно
- *Удобство отладки* — легче найти ошибку в изолированном блоке
- *Абстракция* — можно использовать функцию, не вникая в детали ее реализации

# Основы работы с функциями

В Python функция создается с помощью ключевого слова def (от англ. define — определять). Общий синтаксис выглядит следующим образом :

```
def имя_функции(параметры):
    """Документационная строка (необязательно)"""
    # Тело функции
    # Инструкции
    return результат
```

Правила для имени функции такие же, как и для переменных: оно может содержать строчные буквы, цифры и знаки подчёркивания, но не может начинаться с цифры.

```
def greet():
    print("Привет, мир!")

# Вызов функции
greet()  # Выведет: Привет, мир!
```

# Основы работы с функциями

| Элемент | Описание |
|---|---|
| def | Ключевое слово, сигнализирующее о начале определения функции |
| имя_функции | Идентификатор, по которому функция будет вызываться |
| параметры | Переменные в скобках, через которые функция получает входные данные |
| : | Двоеточие завершает строку определения |
| Документация | Строка в тройных кавычках, описывающая назначение функции |
| Тело функции | Блок кода с отступом, выполняющий основную работу |
| return | Оператор для возврата результата (может отсутствовать) |

# Основы работы с функциями

**Простейшая функция без параметров**

Рассмотрим функцию, которая не принимает параметров и не возвращает значения, а просто выполняет действие:

```
def show_menu():
    print("=== Меню программы ===")
    print("1. Начать игру")
    print("2. Правила")
    print("3. Выход")

# Использование
show_menu()  # При каждом вызове будет выводиться меню
```

Такие функции удобны для вывода часто используемой информации или выполнения повторяющихся действий без необходимости передавать данные.

# Основы работы с функциями

**Функции с параметрами**

Гораздо интереснее работают функции, которые принимают параметры — переменные, через которые в функцию передаются значения для обработки.

***Пример функции с одним параметром:***

```
def greet_user(name):
    print(f"Привет, {name}!")

greet_user("Анна")    # Выведет: Привет, Анна!
greet_user("Иван")    # Выведет: Привет, Иван!
```

# Основы работы с функциями

**Функции с параметрами**

***Пример функции с двумя параметрами:***

```
def print_sum(a, b):
    print(f"Сумма {a} и {b} равна {a + b}")

print_sum(5, 3)       # Выведет: Сумма 5 и 3 равна 8
print_sum(10, 25)     # Выведет: Сумма 10 и 25 равна 35
```

Параметры внутри функции называются формальными параметрами, а конкретные значения, которые передаются при вызове — аргументами

# Основы работы с функциями

**Возврат значений: оператор return**

Функции могут не только выполнять действия, но и возвращать результаты вычислений с помощью оператора return.

```
def add(a, b):
    result = a + b
    return result

# Сохраняем результат в переменную
summa = add(5, 3)
print(summa)  # 8

# Используем результат непосредственно
print(add(10, 20) * 2)  # 60
```

# Основы работы с функциями

**Возврат значений: оператор return**

Важные особенности оператора return :

- *Завершение работы функции* — после выполнения return функция немедленно прекращает работу
- *Возврат значения* — после return можно указать возвращаемое значение
- *Несколько return* — в функции может быть несколько операторов return для разных ветвей выполнения

Пример с несколькими точками возврата:

```
def only_even(numbers):
    for x in numbers:
        if x % 2 != 0:
            return False  # Нашли нечётное — сразу возвращаем False
    return True  # Все числа чётные

print(only_even([2, 4, 6]))  # True
print(only_even([2, 3, 6]))  # False
```

# Основы работы с функциями

**Возврат значений: оператор return**

Если в функции нет оператора return или используется return без значения, функция возвращает специальное значение None (ничего) :

```
def do_nothing():
    pass

result = do_nothing()
print(result)  # None

# Функция print() тоже возвращает None
print(print("Привет"))  # Привет\nNone
```

**Возврат нескольких значений**

В Python функция может вернуть сразу несколько значений. Для этого их нужно перечислить через запятую после return. В программу эти значения вернутся в виде кортежа .

```
def get_stats(numbers):
    total = sum(numbers)
    count = len(numbers)
    average = total / count
    return total, count, average  # Возвращаем три значения

# Результат можно принять как кортеж
stats = get_stats([1, 2, 3, 4, 5])
print(stats)  # (15, 5, 3.0)

# Или распаковать в отдельные переменные
total, count, avg = get_stats([1, 2, 3, 4, 5])
print(f"Сумма: {total}, количество: {count}, среднее: {avg}")
```

# Основы работы с функциями

**Практический пример: проверка строк**

Рассмотрим ситуацию: у нас есть несколько строк с репликами персонажей игры, и нужно проверить, оканчиваются ли они знаком препинания и не являются ли пустыми .

Без функций код выглядел бы громоздко и повторялся:

```
mage = "I use my best spell for protection."
archer = "My people are ready! What's next?"
enemy = "AAAAaaarrrrrggghhhhh"

# Проверка для Mage
if len(mage) == 0:
    print("Строка пуста!")
else:
    if mage[-1] in "!.?":
        print(True)
    else:
        print(False)
```

**Практический пример: проверка строк**

С функцией код становится компактным и понятным:

```
def exam_string(string):
    """Проверяет, что строка не пуста и оканчивается знаком препинания"""
    if len(string) == 0:
        print("Строка пуста!")
        return None
    else:
        return string[-1] in "!.?"

mage = "I use my best spell for protection."
archer = "My people are ready! What's next?"
enemy = "AAAAaaarrrrrggghhhhh"

print(exam_string(mage))   # True
print(exam_string(archer)) # True
print(exam_string(enemy))  # False
```

# Способы передачи аргументов

Python предоставляет гибкие механизмы для передачи аргументов в функции. Рассмотрим основные способы.

**Позиционные аргументы**

При вызове функции аргументы передаются в том же порядке, в котором определены параметры. Это самый простой и распространённый способ .

```
def user_info(name, age, city):
    print(f"Имя: {name}, Возраст: {age}, Город: {city}")

# Позиционная передача — порядок важен!
user_info("Анна", 25, "Москва")  # Имя: Анна, Возраст: 25, Город: Москва
user_info("Москва", 25, "Анна")  # Ошибка логики — порядок нарушен!
```

# Способы передачи аргументов

**Именованные аргументы**

При вызове можно явно указывать, какому параметру соответствует передаваемое значение. В этом случае порядок аргументов не важен.

```
def user_info(name, age, city):
    print(f"Имя: {name}, Возраст: {age}, Город: {city}")

# Именованная передача — порядок не важен
user_info(age=25, city="Москва", name="Анна")
# Имя: Анна, Возраст: 25, Город: Москва
```

Именованные аргументы делают код более читаемым и позволяют пропускать необязательные параметры (если они есть).

# Способы передачи аргументов

**Параметры со значениями по умолчанию**

При определении функции можно задать для параметров значения по умолчанию. Если при вызове аргумент не передан, используется значение по умолчанию.

```
def greet(name, greeting="Здравствуйте"):
    print(f"{greeting}, {name}!")

greet("Анна")                      # Здравствуйте, Анна!
greet("Иван", "Привет")            # Привет, Иван!
greet(name="Пётр", greeting="Добрый вечер")  # Добрый вечер, Пётр!
```

Важное правило: параметры со значениями по умолчанию должны следовать после всех обычных параметров.

# Способы передачи аргументов

**Комбинирование различных типов аргументов**

При вызове функции можно комбинировать позиционные и именованные аргументы. При этом сначала должны идти все позиционные аргументы, затем именованные.

```
def describe_pet(name, animal_type="собака", age=None):
    print(f"Это {animal_type} по имени {name}")
    if age:
        print(f"Возраст: {age} лет")

# Разные способы вызова
describe_pet("Рекс")                          # позиционный + значения по
умолчанию
describe_pet("Мурка", "кошка")                 # два позиционных
describe_pet("Кеша", animal_type="попугай")     # позиционный + именованный
describe_pet(animal_type="хомяк", name="Хома") # все именованные
```

# Способы передачи аргументов

**Изменяемые и неизменяемые аргументы**

Важно понимать, как Python передаёт аргументы в функцию. В Python всё является объектами, и параметры функции получают ссылки на эти объекты. Однако поведение зависит от того, является ли объект изменяемым (mutable) или неизменяемым (immutable).

Неизменяемые типы: числа, строки, кортежи, булевы значения. При попытке изменить такой объект внутри функции создаётся новый объект, а исходный не меняется.

```
def change_number(x):
    x = x + 10
    print(f"Внутри функции: {x}")  # 15

num = 5
change_number(num)
print(f"Снаружи функции: {num}")   # 5 — не изменилось!
```

Изменяемые типы: списки, словари, множества. При изменении такого объекта внутри функции изменения отражаются на исходном объекте.

# Способы передачи аргументов

## Изменяемые и неизменяемые аргументы

Изменяемые типы: списки, словари, множества. При изменении такого объекта внутри функции изменения отражаются на исходном объекте.

```
def add_element(lst):
    lst.append(100)
    print(f"Внутри функции: {lst}")  # [1, 2, 3, 100]

my_list = [1, 2, 3]
add_element(my_list)
print(f"Снаружи функции: {my_list}")  # [1, 2, 3, 100] — изменилось!
```

Однако если внутри функции присвоить параметру новое значение (даже для изменяемого типа), связь с исходным объектом теряется:

```
def reassign_list(lst):
    lst = [100, 200, 300]  # Создаём новый локальный список
    print(f"Внутри функции: {lst}")  # [100, 200, 300]

my_list = [1, 2, 3]
reassign_list(my_list)
print(f"Снаружи функции: {my_list}")  # [1, 2, 3] — не изменилось!
```

# Области видимости переменных

**Область видимости (scope)** определяет, в каких частях программы доступна переменная. Python использует концепцию пространств имён и правила поиска переменных .

**Переменные**, созданные внутри функции, называются локальными. Они существуют только во время выполнения функции и доступны только внутри неё .

```
def my_func():
    y = 10  # локальная переменная
    print(f"Внутри функции: {y}")

my_func()  # Внутри функции: 10
# print(y)  # Ошибка! NameError: name 'y' is not defined
```

# Области видимости переменных

**Глобальные переменные**

Переменные, созданные вне всех функций (на уровне модуля), называются глобальными. Они доступны в любой части программы после своего определения.

```
x = 5  # глобальная переменная

def show_global():
    print(f"Внутри функции: {x}")  # можно читать глобальные переменные

show_global()  # Внутри функции: 5
print(f"Снаружи: {x}")  # Снаружи: 5
```

# Области видимости переменных

**Правило LEGB**

Для разрешения имён Python использует правило LEGB, определяющее порядок поиска переменных:

| Уровень | Название | Описание |
|---|---|---|
| L | Local (локальный) | Переменные внутри текущей функции |
| E | Enclosing (объемлющий) | Переменные функций, внутри которых определена текущая |
| G | Global (глобальный) | Переменные на уровне модуля |
| B | Built-in (встроенный) | Встроенные имена Python (print, len, range и т.д.) |

# Области видимости переменных

**Правило LEGB**

Рассмотрим пример с вложенными функциями:

```
x = "глобальная"

def outer():
    x = "внешняя"
    def inner():
        x = "внутренняя"
        print(x)  # Поиск начинается с Local — находит "внутренняя"
    inner()
    print(x)  # Поиск в outer — находит "внешняя"

outer()
print(x)  # Поиск в глобальной области — находит "глобальная"
```

# Области видимости переменных

**Изменение глобальных переменных: global**

При попытке присвоить значение глобальной переменной внутри функции без специального указания, Python создаёт новую локальную переменную с тем же именем, не затрагивая глобальную.

```
x = 10  # глобальная

def change_x():
    x = 20  # создаётся локальная переменная!
    print(f"Внутри функции: {x}")  # 20

change_x()
print(f"Снаружи: {x}")  # 10 — глобальная не изменилась!
```

# Области видимости переменных

**Изменение глобальных переменных: global**

Чтобы изменить глобальную переменную, нужно объявить её с ключевым словом global внутри функции.

```
x = 10  # глобальная

def change_x():
    global x  # объявляем, что будем работать с глобальной x
    x = 20
    print(f"Внутри функции: {x}")  # 20

change_x()
print(f"Снаружи: {x}")  # 20 — глобальная изменилась!
```

Чрезмерное использование глобальных переменных усложняет анализ программы, делает её менее предсказуемой и затрудняет отладку. Лучше передавать значения через параметры и возвращаемые значения.

# Области видимости переменных

**Нелокальные переменные: nonlocal**

Для работы с переменными из внешней, но не глобальной области видимости (например, во вложенных функциях) используется ключевое слово nonlocal.

```
def outer():
    count = 0

    def inner():
        nonlocal count  # говорим, что count берётся из внешней функции
        count += 1
        return count

    return inner

counter = outer()
print(counter())  # 1
print(counter())  # 2
print(counter())  # 3
```

Без nonlocal во внутренней функции создалась бы локальная переменная count, и счётчик не работал бы.

# Продвинутые концепции

**Функции с произвольным числом аргументов**

Иногда бывает удобно создавать функции, которые могут принимать любое количество аргументов. Для этого используются специальные конструкции: *args и **kwargs.

**Произвольное число позиционных аргументов (*args)**

Символ * перед именем параметра позволяет передавать любое количество позиционных аргументов. Все переданные значения собираются в кортеж.

```
def sum_all(*args):
    print(f"Тип args: {type(args)}")  # <class 'tuple'>
    print(f"Аргументы: {args}")
    return sum(args)

print(sum_all(1, 2, 3))        # Сумма: 6
print(sum_all(5, 10, 15, 20))  # Сумма: 50
print(sum_all(42))              # Сумма: 42
```

**Произвольное число именованных аргументов (**kwargs)**

Два символа ** позволяют принимать произвольное количество именованных аргументов. Они собираются в словарь.

```
def print_info(**kwargs):
    print(f"Тип kwargs: {type(kwargs)}")  # <class 'dict'>
    for key, value in kwargs.items():
        print(f"{key}: {value}")

print_info(name="Анна", age=25, city="Москва")
# name: Анна
# age: 25
# city: Москва

print_info(product="Ноутбук", price=75000, in_stock=True)
```

**Комбинирование различных типов параметров**

При комбинировании разных типов параметров нужно соблюдать порядок :

- Обычные позиционные параметры
- Параметры со значениями по умолчанию
- *args
- Именованные параметры (после *args)
- **kwargs

```
def complex_function(a, b, c=10, *args, option=True, **kwargs):
    print(f"a={a}, b={b}, c={c}")
    print(f"args: {args}")
    print(f"option: {option}")
    print(f"kwargs: {kwargs}")

complex_function(1, 2, 3, 4, 5, 6, option=False, x=100, y=200)
# a=1, b=2, c=3
# args: (4, 5, 6)
# option: False
# kwargs: {'x': 100, 'y': 200}
```

**Документирование функций**

Хорошим тоном является добавление документационной строки (docstring) к каждой создаваемой функции. Она заключается в тройные кавычки и описывает назначение функции, параметры и возвращаемое значение.

```
def factorial(n):
    """Вычисляет факториал числа n (n!).

    Аргументы:
        n (int): Неотрицательное целое число

    Возвращает:
        int: Факториал числа n

    Исключения:
        ValueError: Если n < 0
    """
    if n < 0:
        raise ValueError("Факториал отрицательного числа не определён")
```

## Документирование функций

```
def factorial(n):
    """Вычисляет факториал числа n (n!).

    Аргументы:
        n (int): Неотрицательное целое число

    Возвращает:
        int: Факториал числа n

    Исключения:
        ValueError: Если n < 0
    """
    if n < 0:
        raise ValueError("Факториал отрицательного числа не определён")

    result = 1
    for i in range(1, n + 1):
        result *= i
    return result

# Доступ к документации
print(factorial.__doc__)
help(factorial)
```

# Продвинутые концепции

**Рекурсивные функции**

Рекурсия — это техника программирования, при которой функция вызывает саму себя. Это мощный инструмент для решения задач, которые можно разбить на аналогичные подзадачи.

Структура рекурсивной функции

Любая рекурсивная функция должна содержать два обязательных элемента:

- *Базовый случай (base case)* — условие, при котором рекурсия прекращается
- *Рекурсивный шаг* — вызов функцией самой себя с изменёнными параметрами

**Рекурсивные функции**

Пример: Факториал числа n (обозначается n!) определяется как произведение всех чисел от 1 до n.
Математически:

- 0! = 1 (базовый случай)
- n! = n × (n-1)! (рекурсивный шаг)

```
def factorial(n):
    # Базовый случай
    if n <= 1:
        return 1

    # Рекурсивный шаг
    return n * factorial(n - 1)

print(factorial(5))  # 120
print(factorial(0))  # 1
```

Как это работает:

factorial(5) = 5 * factorial(4)
= 5 * 4 * factorial(3)
= 5 * 4 * 3 * factorial(2)
= 5 * 4 * 3 * 2 * factorial(1)
= 5 * 4 * 3 * 2 * 1 = 120

# Продвинутые концепции

**Последовательность Фибоначчи определяется как:**

- $F(0) = 0$
- $F(1) = 1$
- $F(n) = F(n-1) + F(n-2)$ для $n \geq 2$

```
def fibonacci(n):
    """Возвращает n-е число Фибоначчи."""
    if n <= 0:
        return 0
    elif n == 1:
        return 1
    else:
        return fibonacci(n - 1) + fibonacci(n - 2)

# Первые 10 чисел Фибоначчи
for i in range(10):
    print(f"F({i}) = {fibonacci(i)}")
```

Эта реализация крайне неэффективна (экспоненциальная сложность) из-за множества повторных вычислений. Для практического использования лучше применять итеративный подход.

## Преимущества и недостатки рекурсии

| Преимущества | Недостатки |
|---|---|
| Элегантное решение для задач с рекуррентной природой | Может быть медленной из-за накладных расходов на вызовы |
| Код часто короче и понятнее | Ограничение глубины рекурсии (обычно ~1000) |
| Естественно для работы с рекурсивными структурами данных | Риск переполнения стека при глубокой рекурсии |
| Удобна для обхода деревьев и графов | Итеративные решения часто эффективнее |

**Лямбда-функции**

Лямбда-функции — это анонимные функции, создаваемые с помощью ключевого слова lambda. Они предназначены для простых операций и используются там, где создание полноценной функции было бы избыточным.

Синтаксис:

```
lambda параметры: выражение
```

```
# Обычная функция
def square(x):
    return x ** 2

# Лямбда-функция
square_lambda = lambda x: x ** 2

print(square(5))       # 25
print(square_lambda(5)) # 25

# Лямбда с несколькими параметрами
```

# Продвинутые концепции

**Лямбда-функции**

```
# Обычная функция
def square(x):
    return x ** 2

# Лямбда-функция
square_lambda = lambda x: x ** 2

print(square(5))        # 25
print(square_lambda(5)) # 25

# Лямбда с несколькими параметрами
add = lambda a, b: a + b
print(add(3, 7))  # 10

# Лямбда без параметров
greet = lambda: "Hello, World!"
print(greet())  # Hello, World!
```

**Применение лямбда-функций**

Лямбда-функции особенно полезны в сочетании с функциями высшего порядка, которые принимают другие функции как аргументы.

Сортировка с ключом:

```
students = [
    {"name": "Анна", "grade": 85},
    {"name": "Иван", "grade": 92},
    {"name": "Мария", "grade": 78}
]

# Сортировка по оценкам
sorted_by_grade = sorted(students, key=lambda student: student["grade"])
print(sorted_by_grade)

# Сортировка по длине имени
words = ["python", "java", "c", "javascript"]
sorted_by_len = sorted(words, key=lambda word: len(word))
print(sorted_by_len)  # ['c', 'java', 'python', 'javascript']
```

# Продвинутые концепции

**Применение лямбда-функций**

Функции map(), filter(), reduce():

```
# map — применение функции к каждому элементу
numbers = [1, 2, 3, 4, 5]
squares = list(map(lambda x: x ** 2, numbers))
print(squares)  # [1, 4, 9, 16, 25]

# filter — отбор элементов по условию
evens = list(filter(lambda x: x % 2 == 0, numbers))
print(evens)  # [2, 4]

# reduce — последовательное применение функции
from functools import reduce
product = reduce(lambda x, y: x * y, numbers)
print(product)  # 120 (1*2*3*4*5)
```

# Продвинутые концепции

**Функции как объекты первого класса**

В Python функции являются объектами первого класса (first-class objects). Это означает, что они могут быть:

- Присвоены переменной
- Переданы как аргумент другой функции
- Возвращены из другой функции
- Храниться в структурах данных

# Продвинутые концепции

## Функции как объекты первого класса

```
def shout(text):
    return text.upper()

def whisper(text):
    return text.lower()

# Присваивание переменной
my_func = shout
print(my_func("Hello"))  # HELLO

# Хранение в списке
functions = [shout, whisper]
for func in functions:
    print(func("Hello"))  # HELLO\nhello

# Передача как аргумент
def apply(func, value):
    return func(value)

print(apply(shout, "test"))   # TEST
print(apply(whisper, "TEST")) # test

# Возврат из функции
def get_operation(operation):
    if operation == "double":
        return lambda x: x * 2
    elif operation == "square":
        return lambda x: x ** 2

double = get_operation("double")
print(double(5))  # 10

square = get_operation("square")
print(square(5))  # 25
```

# Заключение

Функцию стоит создавать, когда:

- Повторяется один и тот же код — выносите его в функцию
- Задача может быть решена независимо — функция должна делать одно дело
- Нужно абстрагироваться от деталей — функция скрывает сложность реализации
- Код становится слишком длинным — разбивайте на логические блоки

# Заключение

**Принципы хорошего дизайна функций:**

*1. Единственная ответственность (Single Responsibility)*

Функция должна делать только одно дело, но делать его хорошо.

```
# Плохо — функция делает слишком много
def process_user(name, age, email):
    # Проверяет данные
    if not email or age < 0:
        return False
    # Сохраняет в БД
    # Отправляет приветственное письмо
    # Возвращает результат
    pass

# Хорошо — каждая функция отвечает за одно
def validate_user_data(name, age, email):
    return email and age >= 0

def save_user(name, age, email):
    pass

def send_welcome_email(email):
    pass
```

# Заключение

**Принципы хорошего дизайна функций:**

*2. Читаемость и понятность*

Имя функции должно ясно описывать, что она делает .

```
# Плохо
def f(a, b):
    return a * b

# Хорошо
def calculate_rectangle_area(width, height):
    return width * height
```

# Заключение

**Принципы хорошего дизайна функций:**

*3. Избегайте побочных эффектов*

Функция должна, по возможности, не изменять глобальные переменные и переданные аргументы (если это не ожидается).

```
# Плохо — функция меняет переданный список
def add_item_bad(item, lst):
    lst.append(item)

# Лучше — возвращает новый список
def add_item_good(item, lst):
    return lst + [item]
```

*4. Документируйте функции*

Всегда добавляйте docstring для нетривиальных функций

# Заключение

**Типичные ошибки при работе с функциями**

*Ошибка 1: Забытый return*

```
def add(a, b):
    result = a + b
    # забыли return

x = add(5, 3)
print(x)  # None — неожиданно!
```

*Ошибка 2: Изменение глобальных переменных по неосторожности*

```
counter = 0

def increment():
    counter += 1  # Ошибка! UnboundLocalError

# Нужно объявить global или вернуть значение
```

# Заключение

**Типичные ошибки при работе с функциями**

*Ошибка 3: Использование изменяемых объектов как значений по умолчанию*

```
def add_item(item, lst=[]):  # Плохо! Список создаётся один раз
    lst.append(item)
    return lst

print(add_item(1))  # [1]
print(add_item(2))  # [1, 2] — неожиданно!

# Правильно
def add_item(item, lst=None):
    if lst is None:
        lst = []
    lst.append(item)
    return lst
```

# Заключение

**Типичные ошибки при работе с функциями**

*Ошибка 4: Слишком глубокая рекурсия без базового случая*

```
def infinite_recursion():
    return infinite_recursion()  # RecursionError!
```

# Заключение

**Функции — это фундаментальный инструмент структурирования кода в Python. Они позволяют:**

- Разбивать сложные задачи на простые, понятные блоки
- Избегать дублирования кода
- Создавать переиспользуемые компоненты
- Скрывать детали реализации за понятными именами

# Заключение

**Ключевые выводы:**

- Функции создаются с помощью ключевого слова def и могут принимать параметры и возвращать значения
- return возвращает результат и завершает работу функции; если return нет, функция возвращает None
- Аргументы можно передавать позиционно или по имени
- Параметры могут иметь значения по умолчанию
- Переменные внутри функции — локальные, они не видны снаружи
- Для изменения глобальных переменных используется global
- *args и **kwargs позволяют создавать гибкие функции с произвольным числом аргументов
- Рекурсия — мощный инструмент для задач с самоподобием
- Лямбда-функции удобны для простых операций
- В Python функции — объекты первого класса, их можно присваивать, передавать и возвращать